



IA CORPORATIVA COM JAVA E LANGCHAIN4J

AULA 09 - CAPACIDADES DE AGENTES COM
TOOLS



OBJETIVOS DA AULA

- ❑ Compreender a anatomia de uma Tool em Langchain4j e a importância crítica de sua descrição como um contrato de API para o LLM.
- ❑ Simular um serviço de backend (BookingService) para representar o sistema da agência de viagens.
- ❑ Implementar uma classe de Tools (BookingTools) que expõe as capacidades de negócio do backend para o agente.
- ❑ Projetar Tools que lidam com parâmetros, retornam objetos estruturados e gerenciam diferentes cenários de negócio (sucesso, erro, não encontrado).
- ❑ Discutir as melhores práticas e padrões de design para a criação de Tools robustas e fáceis de serem compreendidas pelo LLM.



A ANATOMIA DE UMA TOOL

A ANATOMIA DE UMA TOOL

- ❑ Um agente decide qual ação tomar com base em um "manifesto" de Tools que ele conhece.
 - ❑ No Langchain4j, esse manifesto é gerado a partir de métodos Java anotados com @Tool.
- ❑ O que é uma Tool?
 - ❑ É um método Java, dentro de um bean CDI, que expõe uma capacidade específica do seu sistema para o LLM.
- ❑ A Descrição é a Especificação da API:
 - ❑ A parte mais importante da anotação @Tool é sua descrição em texto.
 - ❑ Esta descrição não é um comentário para documentação. É a especificação da API que o LLM irá ler para tomar uma decisão.
 - ❑ Uma descrição ambígua ou incompleta levará o LLM a usar a Tool de forma incorreta ou a não usá-la.

BOAS PRÁTICAS PARA DESCRIÇÕES DE TOOLS

- ❑ A qualidade da descrição impacta diretamente a autonomia e a precisão do agente. Pense nela como a documentação de uma API REST para um cliente não-humano.

Má Prática	Boa Prática	Análise Técnica
@Tool("Gerencia reservas")	@Tool("Obtém os detalhes completos de uma reserva com base em seu número de identificação (bookingId).")	A má prática é ambígua. "Gerenciar" como? A boa prática é explícita sobre a ação (obter detalhes) e o parâmetro chave (bookingId).
@Tool("Cancela a reserva")	@Tool("Cancela uma reserva existente. Para confirmar o cancelamento, é necessário fornecer o ID da reserva (bookingId) e o último nome do cliente (customerLastName).")	A má prática omite os parâmetros necessários. A boa prática informa ao LLM exatamente quais informações ele precisa extrair do prompt do usuário para executar a ação com sucesso.
@Tool("Busca carros")	@Tool("Encontra carros disponíveis para aluguel em um local específico para uma data de início e uma data de fim. Retorna uma lista de modelos de carros disponíveis.")	A má prática é vaga. A boa prática especifica os três parâmetros necessários (local, data de início, data de fim) e o que esperar como retorno, ajudando o LLM a planejar os próximos passos.

SIMULAÇÃO DO BACKEND: O BOOKINGSERVICE



SIMULAÇÃO DO BACKEND: O BOOKINGSERVICE

- ❑ Nossas Tools precisam interagir com um sistema real. Para simular isso, criaremos um serviço de backend que gerencia as reservas em memória. Isso nos permite focar na lógica do agente sem a complexidade de um banco de dados real.
- ❑ Passo 1: Criar o Modelo de Dados

```
package dev.ia;
```

```
public enum BookingStatus {  
    CONFIRMED,  
    CANCELLED,  
    PENDING  
}
```

```
package dev.ia;
```

```
import java.time.LocalDate;  
  
public record Booking(  
    Long id,  
    String customerName,  
    String destination,  
    LocalDate startDate,  
    LocalDate endDate,  
    BookingStatus status  
) {}
```

SIMULAÇÃO DO BACKEND: O BOOKINGSERVICE

❑ Passo 2: Criar o BookingService

- ❑ Este bean @ApplicationScoped atuará como nosso sistema de gerenciamento de reservas.

```
package dev.ia;

import jakarta.enterprise.context.ApplicationScoped;
import java.time.LocalDate;
import java.util.*;

@ApplicationScoped
public class BookingService {

    private final Map<Long, Booking> bookings = new HashMap<>();

    public BookingService() {
        bookings.put(12345L, new Booking(12345L, "John Doe", "Tesouros do Egito",
            LocalDate.now().plusMonths(2), LocalDate.now().plusMonths(2).plusDays(10), BookingStatus.CONFIRMED));
        bookings.put(67890L, new Booking(67890L, "Jane Smith", "Aventura Amazônia",
            LocalDate.now().plusMonths(3), LocalDate.now().plusMonths(3).plusDays(7), BookingStatus.CONFIRMED));
    }

    public Optional<Booking> getBookingDetails(long bookingId) {
        return Optional.ofNullable(bookings.get(bookingId));
    }

    public Optional<Booking> cancelBooking(long bookingId, String customerLastName) {
        if (bookings.containsKey(bookingId)) {
            Booking booking = bookings.get(bookingId);
            if (booking.customerName().endsWith(customerLastName)) {
                Booking cancelledBooking = new Booking(booking.id(), booking.customerName(), booking.destination(),
                    booking.startDate(), booking.endDate(), BookingStatus.CANCELLED);
                bookings.put(bookingId, cancelledBooking);
                return Optional.of(cancelledBooking);
            }
        }
        return Optional.empty();
    }
}
```



SIMULAÇÃO DO BACKEND: O BOOKINGSERVICE

❑ Passo 3:

- ❑ Agora, vamos criar a classe que conterà os métodos que o agente poderá chamar.

```
package dev.ia;

import dev.langchain4j.agent.tool.Tool;
import jakarta.enterprise.context.ApplicationScoped;
import jakarta.inject.Inject;

@ApplicationScoped
public class BookingTools {

    @Inject
    BookingService bookingService;

    @Tool("Obtém os detalhes completos de uma reserva com base em seu número de identificação (bookingId).")
    public String getBookingDetails(long bookingId) {
        return bookingService.getBookingDetails(bookingId)
            .map(Booking::toString)
            .orElse("Reserva com ID " + bookingId + " não encontrada.");
    }
}
```

SIMULAÇÃO DO BACKEND: O BOOKINGSERVICE

❑ Passo 4:

- ❑ Vamos adicionar o método para cancelar uma reserva. Aqui, o tratamento de diferentes resultados (sucesso vs. não encontrado) é crucial.

```
// Dentro da classe BookingTools
```

```
@Tool("""
```

```
    Cancela uma reserva existente.
```

```
    Para confirmar o cancelamento, é necessário fornecer o ID da reserva (bookingId)
```

```
    e o último nome do cliente (customerLastName).
```

```
    """)
```

```
public String cancelBooking(long bookingId, String customerLastName) {
```

```
    return bookingService.cancelBooking(bookingId, customerLastName)
```

```
        .map(booking -> "Reserva " + bookingId + " cancelada com sucesso. Status atual: " + booking.status())
```

```
        .orElse("Não foi possível cancelar a reserva. Verifique se o ID da reserva e o sobrenome do cliente estão corretos.");
```

```
}
```

DESIGN DE TOOLS: PADRÕES E MELHORES PRÁTICAS

DESIGN DE TOOLS: PADRÕES E MELHORES PRÁTICAS

- ❑ Projetar Tools é projetar uma API para um LLM. Devemos seguir princípios de design robustos.
- ❑ Single Responsibility Principle:
 - ❑ Cada Tool deve ter uma única e clara responsabilidade.
 - ❑ Anti-pattern: Criar uma Tool genérica como `manageBooking(String action, long bookingId)`. Isso força o LLM a "adivinhar" a string de ação correta ("CANCEL", "GET", "UPDATE") e torna a descrição da Tool confusa.
 - ❑ Boa prática: Criar Tools específicas como `cancelBooking(long bookingId)` e `getBookingDetails(long bookingId)`. Isso resulta em descrições claras e um mapeamento de intenção muito mais confiável.
- ❑ Retornos Descritivos e Completos:
 - ❑ O valor de retorno de uma Tool é a única fonte de informação para o próximo ciclo de pensamento do LLM.
 - ❑ Anti-pattern: Retornar boolean ou void. Um `true` não informa ao LLM o que aconteceu.
 - ❑ Boa prática: Retornar uma String ou um DTO (Data Transfer Object) que descreva o resultado da operação. Ex: "Reserva 12345 cancelada com sucesso. Um e-mail de confirmação foi enviado." é muito mais útil para o LLM do que `true`.

DESIGN DE TOOLS: PADRÕES E MELHORES PRÁTICAS

- ❑ Erros de Negócio como Fluxo, não Exceções:
 - ❑ Como discutido anteriormente, uma exceção não tratada quebra o ciclo de raciocínio do agente.
 - ❑ Anti-pattern:

```
@Tool("...")  
  
public Booking getBookingDetails(long id) {  
    // Lança BookingNotFoundException se não encontrar  
    return bookingService.findByIdOrThrow(id);  
}
```

DESIGN DE TOOLS: PADRÕES E MELHORES PRÁTICAS

- ❑ Boa prática: Capturar exceções de negócio ou usar Optional para transformar o estado de erro em uma mensagem de retorno informativa que o LLM possa usar.

```
@Tool("...")
```

```
public String getBookingDetails(long id) {  
    return bookingService.getBookingDetails(id)  
        .map(Booking::toString)  
        .orElse("Erro: A reserva com o ID " + id + " não foi encontrada.");  
}
```

RESUMO E PRÓXIMOS PASSOS

RESUMO E PRÓXIMOS PASSOS

- ❑ O que Construímos Hoje?
 - ❑ Aprofundamos nosso entendimento sobre a @Tool como uma especificação de API para o LLM, com foco na importância de descrições claras e explícitas.
 - ❑ Criamos um modelo de dados (Booking, BookingStatus) e um serviço de backend (BookingService) para simular um sistema de negócio real.
 - ❑ Implementamos a classe BookingTools, que atua como uma camada de API entre o agente e nossos serviços de negócio.
 - ❑ Discutimos e aplicamos padrões de design para Tools, como o Princípio da Responsabilidade Única e o tratamento de erros como parte do fluxo de conversação.
- ❑ Próxima Aula:
 - ❑ Com nossas Tools prontas, o próximo passo é conectá-las ao nosso agente.
 - ❑ Na sequência, vamos modificar nosso AiService para que ele conheça a BookingTools, configurar a memória da conversa (essencial para agentes) e realizar os primeiros testes de ponta a ponta, enviando prompts que exigem a execução das Tools que acabamos de criar.

ATÉ A PRÓXIMA!